PROGRAM 1 `1_matrix_add.c`

Matrix Addition

C SOURCE CODE

1_matrix_add.c

```c
/* Matrix Addition */

#include <stdio.h>

int main() {
    int row, col;

    printf("Enter row & column sizes of 1st matrix: ");
    scanf("%d%d", &row, &col);
    int A[row][col], B[row][col];

    for (int count = 1; count < 3; count++) {
        const char *matrix_number = (count == 1) ? "1st" : "2nd";
        int (*matrix)[col] = (count == 1) ? A : B;
        printf("Enter %d elemnts for %s matrix: ", row*col, matrix_number);

        for (int i=0; i < row; i++)
            for (int j=0; j < col; j++)
                scanf("%d", &matrix[i][j]);
    }

    printf("Added:\n");
    for (int i=0; i < row; i++) {
        for (int j=0; j < col; j++)
            printf("%d ", A[i][j] + B[i][j]);
        printf("\n");
    }

    return 0;
}
```

PROGRAM OUTPUT

Execution Verification

```
Enter row & column sizes of 1st matrix: 2 2
Enter 4 elemnts for 1st matrix: 1 2 3 4
Enter 4 elemnts for 2nd matrix: 5 6 7 8
Added:
6 8
10 12
```

PROGRAM 2 2_polynomial_add.c

Addition of Two Polynomials Using Structures

C SOURCE CODE 2_polynomial_add.c

```c
// Addition of 2 polynomials using structure

#include <stdio.h>

struct Poly {
    int coeff, exp;
};


int main() {
    int size_1, size_2;

    printf("Enter number of elements for 1st Polynomial: ");
    scanf("%d", &size_1);
    printf("Enter number of elements for 2nd Polynomial: ");
    scanf("%d", &size_2);

    struct Poly A[size_1], B[size_2], SUM[size_1+size_2];

    for (int count = 1; count < 3; count++) {
        const char *polynomial_number = (count == 1) ? "1st" : "2nd";
        struct Poly *polynomial = (count == 1) ? A : B;
        int size = (count == 1) ? size_1 : size_2;
        printf("\nEnter %d elements for %s polynomial:\n", size, polynomial_number);

        for (int i=0; i < size; i++)
            scanf("%d%d", &polynomial[i].coeff, &polynomial[i].exp);
    }

    int i = 0, j = 0, k = 0;
    while (i < size_1 && j < size_2) {
        if (A[i].exp == B[j].exp) {
            SUM[k].coeff = A[i].coeff + B[j].coeff;
            SUM[k].exp = A[i].exp;
            i++; j++;
        } else if (A[i].exp > B[j].exp) {
            SUM[k] = A[i];
            i++;
        } else {
            SUM[k] = B[j];
            j++;
        }
        k++;
    }

    for (i; i < size_1; i++, k++) SUM[k] = A[i];
    for (j; j < size_2; j++, k++) SUM[k] = B[j];

    printf("Sum:\n");
    for (i = 0; i < k; i++) {
        printf("(%d**%d)", SUM[i].coeff, SUM[i].exp);
        if (i < k-1) printf(" + ");
    }

    return 0;
}
```

PROGRAM OUTPUT Execution Verification

```
Enter number of elements for 1st Polynomial: 2
Enter number of elements for 2nd Polynomial: 2

Enter 2 elements for 1st polynomial:
4 2
```

```
3 1

Enter 2 elements for 2nd polynomial:
5 2
2 0
Sum:
(9**2) + (3**1) + (2**0)
```

PROGRAM 3 3_matrix_to_sparse.c

Tuple Representation of Sparse Matrix

C SOURCE CODE 3_matrix_to_sparse.c

```c
/* Tuple Representation of Sparse Matrix */

#include <stdio.h>

int main() {
    int row, col, number_non_zero = 0;
    printf("Enter row & column size: ");
    scanf("%d%d", &row, &col);

    int matrix[row][col];
    printf("Enter %d elements: ", row*col);
    for (int i = 0; i < row; i++)
        for (int j = 0; j < col; j++) {
            scanf("%d", &matrix[i][j]);
    }

    int sparse[row*col + 1][3], r = 1;
    sparse[0][0] = row;
    sparse[0][1] = col;

    for (int i = 0; i < row; i++)
        for (int j = 0; j < col; j++)
            if (matrix[i][j] != 0) {
                sparse[r][0] = i;
                sparse[r][1] = j;
                sparse[r][2] = matrix[i][j];
                r++;
                number_non_zero++;
            }

    sparse[0][2] = number_non_zero;

    for (int i = 0; i ≤ number_non_zero; i++) {
        printf("(");
        for (int j = 0; j < 3; j++) {
            printf("%d", sparse[i][j]);
            if (j < 2) printf(", ");
        }
        printf(")\n");
    }

    return 0;
}
```

PROGRAM OUTPUT Execution Verification

```
Enter row & column size: 3 3
Enter 9 elements: 0 0 5 0 8 0 0 0 0
(3, 3, 2)
(0, 2, 5)
(1, 1, 8)
```

PROGRAM 4 `4_sparse_matrix_add.c`

Sparse Matrix Addition

C SOURCE CODE 4_sparse_matrix_add.c

```c
/* Sparse Matrix Addition */

#include <stdio.h>

int main() {
    int rows, cols, non_zero1, non_zero2;
    printf("Enter total number of rows and columns for the matrices: ");
    scanf("%d%d", &rows, &cols);
    printf("Enter number of non-zero elements for Matrix 1: ");
    scanf("%d", &non_zero1);
    printf("Enter number of non-zero elements for Matrix 2: ");
    scanf("%d", &non_zero2);

    int sparse1[non_zero1 + 1][3];
    int sparse2[non_zero2 + 1][3];
    int sum[non_zero1 + non_zero2 + 1][3];

    sparse1[0][0] = rows; sparse1[0][1] = cols; sparse1[0][2] = non_zero1;
    sparse2[0][0] = rows; sparse2[0][1] = cols; sparse2[0][2] = non_zero2;
    sum[0][0] = rows; sum[0][1] = cols;

    printf("\nEnter Triplets for Matrix 1 (Row, Column, Value):\n");
    for (int i = 1; i ≤ non_zero1; i++) {
        scanf("%d%d%d", &sparse1[i][0], &sparse1[i][1], &sparse1[i][2]);
    }

    printf("\nEnter Triplets for Matrix 2 (Row, Column, Value):\n");
    for (int i = 1; i ≤ non_zero2; i++) {
        scanf("%d%d%d", &sparse2[i][0], &sparse2[i][1], &sparse2[i][2]);
    }

    int i = 1, j = 1, k = 1;

    while (i ≤ non_zero1 && j ≤ non_zero2) {
        if (sparse1[i][0] == sparse2[j][0]) {
            if (sparse1[i][1] == sparse2[j][1]) {
                sum[k][0] = sparse1[i][0];
                sum[k][1] = sparse1[i][1];
                sum[k][2] = sparse1[i][2] + sparse2[j][2];
                i++; j++; k++;
            }
            else if (sparse1[i][1] < sparse2[j][1]) {
                sum[k][0] = sparse1[i][0];
                sum[k][1] = sparse1[i][1];
                sum[k][2] = sparse1[i][2];
                i++; k++;
            }
            else {
                sum[k][0] = sparse2[j][0];
                sum[k][1] = sparse2[j][1];
                sum[k][2] = sparse2[j][2];
                j++; k++;
            }
        }
        else if (sparse1[i][0] < sparse2[j][0]) {
            sum[k][0] = sparse1[i][0];
            sum[k][1] = sparse1[i][1];
            sum[k][2] = sparse1[i][2];
            i++; k++;
        }
        else {
            sum[k][0] = sparse2[j][0];
            sum[k][1] = sparse2[j][1];
            sum[k][2] = sparse2[j][2];
            j++; k++;
```

```c
        }
    }

    while (i ≤ non_zero1) {
        sum[k][0] = sparse1[i][0];
        sum[k][1] = sparse1[i][1];
        sum[k][2] = sparse1[i][2];
        i++; k++;
    }

    while (j ≤ non_zero2) {
        sum[k][0] = sparse2[j][0];
        sum[k][1] = sparse2[j][1];
        sum[k][2] = sparse2[j][2];
        j++; k++;
    }

    sum[0][2] = k - 1;

    printf("\nResultant Sparse Matrix (Triplet Representation):\n");
    printf("Row\tCol\tValue\n");
    for (int m = 0; m < k; m++) {
        printf("%d\t%d\t%d\n", sum[m][0], sum[m][1], sum[m][2]);
    }

    return 0;
}
```

PROGRAM OUTPUT Execution Verification

```
Enter total number of rows and columns for the matrices: 3 3
Enter number of non-zero elements for Matrix 1: 2
Enter number of non-zero elements for Matrix 2: 2

Enter Triplets for Matrix 1 (Row, Column, Value):
0 0 5
1 1 3

Enter Triplets for Matrix 2 (Row, Column, Value):
0 0 2
2 2 4

Resultant Sparse Matrix (Triplet Representation):
Row     Col     Value
3       3       3
0       0       7
1       1       3
2       2       4
```

PROGRAM 5 `5_sparse_trans.c`

Transpose of Sparse Matrix

C SOURCE CODE 5_sparse_trans.c

```c
/* transpose of sparse matrix */

#include <stdio.h>

int main() {
    int rows, cols, non_zero;
    printf("Enter total number of rows and columns for the matrix: ");
    scanf("%d%d", &rows, &cols);
    printf("Enter number of non-zero elements for the matrix: ");
    scanf("%d", &non_zero);

    int sparse[non_zero + 1][3];
    sparse[0][0] = rows; sparse[0][1] = cols; sparse[0][2] = non_zero;

    printf("\nEnter Triplets for the Matrix (Row, Column, Value):\n");
    for (int i = 1; i ≤ non_zero; i++) {
        scanf("%d%d%d", &sparse[i][0], &sparse[i][1], &sparse[i][2]);
    }

    int transpose[non_zero + 1][3];
    transpose[0][0] = cols; transpose[0][1] = rows; transpose[0][2] = non_zero;

    // Perform transpose
    for (int i = 1; i ≤ non_zero; i++) {
        transpose[i][0] = sparse[i][1]; // Swap row and column
        transpose[i][1] = sparse[i][0];
        transpose[i][2] = sparse[i][2];
    }

    // Sort the transposed matrix based on row and column
    for (int i = 1; i ≤ non_zero; i++) {
        for (int j = i + 1; j ≤ non_zero; j++) {
            if ((transpose[i][0] > transpose[j][0]) ||
                (transpose[i][0] == transpose[j][0] && transpose[i][1] > transpose[j][1])) {

                for (int k = 0; k < 3; k++) {
                    int temp = transpose[i][k];
                    transpose[i][k] = transpose[j][k];
                    transpose[j][k] = temp;
                }
            }
        }
    }

    // Display the transposed matrix
    printf("\nThe transposed matrix is:\n");
    for (int i = 0; i ≤ non_zero; i++) {
        printf("%d %d %d\n", transpose[i][0], transpose[i][1], transpose[i][2]);
    }

    return 0;
}
```

PROGRAM OUTPUT Execution Verification

```
Enter total number of rows and columns for the matrix: 3 3
Enter number of non-zero elements for the matrix: 3

Enter Triplets for the Matrix (Row, Column, Value):
0 1 5
1 2 8
2 0 3

The transposed matrix is:
```

```
3 3 3
0 2 3
1 0 5
2 1 8
```

PROGRAM 6 `6_stack.c` **Stack Operations Using Array**

C SOURCE CODE 6_stack.c

```c
/* Implementation of a stack in C */

#include <stdio.h>
#define MAX 10

struct Stack {
    int top;
    int items[MAX];
};

void initialize(struct Stack *s) {
    s->top = -1;
}

void push(struct Stack *s, int val) {
    if (s->top >= MAX-1) {
        printf("Stack Overflow\n");
        return;
    }
    s->items[++(s->top)] = val;
}

int pop(struct Stack *s) {
    if (s->top == -1) {
        printf("Stack Underflow\n");
        return -1;
    }
    return s->items[s->top--];
}

void display(struct Stack s) {
    if (s.top == -1) {
        printf("Stack Underflow\n");
        return;
    }
    printf("Displaying contents:\n");
    for (int i=0; i<=s.top; i++) {
        printf("%d\n", s.items[i]);
    }
}

int main() {
    struct Stack s;
    initialize(&s);

    int choice, value;

    do {
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to push: ");
                scanf("%d", &value);
                push(&s, value);
                break;
            case 2:
                value = pop(&s);
                if (value != -1) {
                    printf("Popped value: %d\n", value);
```

```c
                }
                break;
            case 3:
                display(s);
                break;
            case 4:
                printf("Exiting ... \n");
                break;
            default:
                printf("Invalid choice. Please try again.\n");
        }
    } while (choice ≠ 4);

    return 0;
}
```

PROGRAM OUTPUT Execution Verification

```
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter value to push: 10
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter value to push: 20
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Displaying contents:
10
20
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 2
Popped value: 20
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Displaying contents:
10
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 4
Exiting ...
```

PROGRAM 7 `7_standard_queue.c`

Linear Queue Operations Using Array

C SOURCE CODE 7_standard_queue.c

```c
/* Implemetation of Standard Queue*/

#include <stdio.h>
#define MAX 3

struct Queue
{
    int front;
    int rear;
    int items[MAX];
};

void initialize(struct Queue *q) {
    q->front = -1;
    q->rear = -1;
}

void enqueu(struct Queue *q, int val) {
    if (q->rear >= MAX-1) {
        printf("Queue Overflow\n");
        return;
    }
    if (q->front == -1) q->front++;
    q->items[++(q->rear)] = val;
}

int dequeu(struct Queue *q) {
    if (q->front == -1 || q->front > q->rear) {
        printf("Queue Underflow\n");
        return -1;
    }
    return q->items[q->front++];
}

void display(struct Queue q) {
    if (q.front == -1 || q.front > q.rear) {
        printf("Queue Underflow\n");
        return;
    }
    printf("Displaying Contents:\n");
    for (int i=q.front; i<=q.rear; i++) {
        printf("%d\n", q.items[i]);
    }
}

int main() {
    struct Queue q;
    initialize(&q);

    int choice, value;

    do {
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to enqueue: ");
                scanf("%d", &value);
                enqueu(&q, value);
                break;
```

```c
            case 2:
                value = dequeu(&q);
                if (value != -1) {
                    printf("Dequeued value: %d\n", value);
                }
                break;
            case 3:
                display(q);
                break;
            case 4:
                printf("Exiting ... \n");
                break;
            default:
                printf("Invalid choice! Please try again.\n");
        }
    } while (choice != 4);

    return 0;
}

```

PROGRAM OUTPUT Execution Verification

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 100
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 200
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Displaying Contents:
100
200
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
Dequeued value: 100
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Displaying Contents:
200
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 4
Exiting ...
```

PROGRAM 8 8_circular_queue.c

Circular Queue Operations Using Array

C SOURCE CODE 8_circular_queue.c

```c
/* Circular Queue Implementation */

#include <stdio.h>
#define MAX 3

struct CircularQueue
{
    int front;
    int rear;
    int count;
    int items[MAX];
};

void initialize(struct CircularQueue *q) {
    q->front = 0;
    q->rear = MAX - 1;
    q->count = 0;
}

void enqueue(struct CircularQueue *q, int val) {
    if (q->count >= MAX) {
        printf("Queue Overflow\n");
        return;
    }
    q->rear = (q->rear+1)%MAX;
    q->items[q->rear] = val;
    q->count++;
}

int dequeue(struct CircularQueue *q) {
    if (q->count == 0) {
        printf("Queue Underflow\n");
        return -1;
    }
    int val = q->items[q->front];
    q->front = (q->front+1)%MAX;
    q->count--;
    return val;
}

void display(struct CircularQueue q) {
    if (q.count == 0) {
        printf("Queue empty\n");
        return;
    }

    printf("Displaying Contents:\n");
    for (int i=q.front; i < q.front+q.count; i++) {
        printf("%d\n", q.items[i%MAX]);
    }
}

int main() {
    struct CircularQueue q;
    initialize(&q);

    int choice, value;

    do {
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
```

```c

        switch (choice) {
            case 1:
                printf("Enter value to enqueue: ");
                scanf("%d", &value);
                enqueue(&q, value);
                break;
            case 2:
                value = dequeue(&q);
                if (value != -1) {
                    printf("Dequeued value: %d\n", value);
                }
                break;
            case 3:
                display(q);
                break;
            case 4:
                printf("Exiting ... \n");
                break;
            default:
                printf("Invalid choice! Please try again.\n");
        }
    } while (choice != 4);

    return 0;
}

```

PROGRAM OUTPUT Execution Verification

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 10
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 20
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 30
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Displaying Contents:
10
20
30
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
Dequeued value: 10
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 40
```

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Displaying Contents:
20
30
40
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 4
Exiting ...
```

PROGRAM 9 `9_dequeue.c`

Double Ended Queue (Deque) Operations

C SOURCE CODE 9_dequeue.c

```c
/* Double Ended Queue Implementation */

#include <stdio.h>
#define MAX 10

struct Dequeue {
    int front;
    int rear;
    int items[MAX];
};

void initializeDeque(struct Dequeue *dq) {
    dq->front = -1;
    dq->rear = -1;
}

void insertFront(struct Dequeue *dq, int val) {
    if (dq->front == 0 && dq->rear == MAX-1) {
        printf("Queue Overflow\n");
        return;
    }

    if (dq->front == -1) {
        dq->items[++dq->front] = val;
        dq->rear++;
    } else if (dq->front > 0)
        dq->items[--dq->front] = val;
    else {
        int i = dq->rear;
        while (i>=dq->front) {
            dq->items[i+1] = dq->items[i];
            i--;
        }
        dq->items[dq->front] = val;
        dq->rear++;
    }
}

void insertRear(struct Dequeue *dq, int val) {
    if (dq->front == 0 && dq->rear == MAX-1) {
        printf("Queue Overflow\n");
        return;
    }

    if (dq->front == -1) {
        dq->items[++dq->front] = val;
        dq->rear++;
    } else if (dq->rear < MAX-1)
        dq->items[++dq->rear] = val;
    else {
        int i = dq->front;
        while (i<=dq->rear) {
            dq->items[i-1] = dq->items[i];
            i++;
        }
        dq->items[dq->rear] = val;
        dq->front--;
    }
}

int removeFront(struct Dequeue *dq) {
    if (dq->front == -1 && dq->rear == -1) {
        printf("Queue Underflow\n");
        return -1;
    }
```

```c
    int val = dq->items[dq->front];

    if (dq->front == dq->rear) {
        dq->front = dq->rear = -1;
    } else
        dq->front++;

    return val;
}

int removeRear(struct Dequeue *dq) {
    if (dq->front == -1 && dq->rear == -1) {
        printf("Queue Underflow\n");
        return -1;
    }
    int val = dq->items[dq->rear];

    if (dq->front == dq->rear) {
        dq->front = dq->rear = -1;
    } else
        dq->rear--;

    return val;
}

void display(struct Dequeue dq) {
    if (dq.front == -1) {
        printf("Queue is Empty\n");
        return;
    }
    printf("Displaying Contents:\n");
    for (int i=dq.front; i<=dq.rear; i++)
        printf("%d\n", dq.items[i]);
}

int main() {
    struct Dequeue dq;
    initializeDeque(&dq);

    int choice, value;

    do {
        printf("1. Insert Front\n");
        printf("2. Insert Rear\n");
        printf("3. Remove Front\n");
        printf("4. Remove Rear\n");
        printf("5. Display\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to insert at front: ");
                scanf("%d", &value);
                insertFront(&dq, value);
                break;
            case 2:
                printf("Enter value to insert at rear: ");
                scanf("%d", &value);
                insertRear(&dq, value);
                break;
            case 3:
                value = removeFront(&dq);
                if (value != -1)
                    printf("Removed from front: %d\n", value);
                break;
            case 4:
                value = removeRear(&dq);
                if (value != -1)
```

```
                    printf("Removed from rear: %d\n", value);
                break;
            case 5:
                display(dq);
                break;
            case 6:
                printf("Exiting ... \n");
                break;
            default:
                printf("Invalid choice! Please try again.\n");
        }
    } while (choice ≠ 6);

    return 0;
}
```

PROGRAM OUTPUT Execution Verification

```
1. Insert Front
2. Insert Rear
3. Remove Front
4. Remove Rear
5. Display
6. Exit
Enter your choice: 2
Enter value to insert at rear: 10
1. Insert Front
2. Insert Rear
3. Remove Front
4. Remove Rear
5. Display
6. Exit
Enter your choice: 2
Enter value to insert at rear: 20
1. Insert Front
2. Insert Rear
3. Remove Front
4. Remove Rear
5. Display
6. Exit
Enter your choice: 1
Enter value to insert at front: 5
1. Insert Front
2. Insert Rear
3. Remove Front
4. Remove Rear
5. Display
6. Exit
Enter your choice: 5
Displaying Contents:
5
10
20
1. Insert Front
2. Insert Rear
3. Remove Front
4. Remove Rear
5. Display
6. Exit
Enter your choice: 3
Removed from front: 5
1. Insert Front
2. Insert Rear
3. Remove Front
4. Remove Rear
5. Display
6. Exit
Enter your choice: 4
```

```
Removed from rear: 20
1. Insert Front
2. Insert Rear
3. Remove Front
4. Remove Rear
5. Display
6. Exit
Enter your choice: 5
Displaying Contents:
10
1. Insert Front
2. Insert Rear
3. Remove Front
4. Remove Rear
5. Display
6. Exit
Enter your choice: 6
Exiting ...
```

PROGRAM 10 `10_infixtopostfix.c`

Infix to Postfix Expression Conversion

C SOURCE CODE 10_infixtopostfix.c

```c
/* Infix to Postfix Conversion */


#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>

#define MAX 50

struct Stack {
    int top;
    char items[MAX];
};

void initialize(struct Stack *s) {
    s->top = -1;
}

void push(struct Stack *s, char val) {
    if (s->top >= MAX-1) return;
    s->items[++(s->top)] = val;
}

char pop(struct Stack *s) {
    if (s->top == -1) return '\0';
    return s->items[s->top--];
}

char peek(struct Stack s) {
    if (s.top == -1) return '\0';
    return s.items[s.top];
}

int precedence(char op) {
    if (op == '^') return 3;
    if (op == '*' || op == '/') return 2;
    if (op == '+' || op == '-') return 1;
    return 0; // '(' or non-operator
}

int main() {
    struct Stack s;
    initialize(&s);
    char infix[50], buffer;
    int i = 0;

    printf("Enter infix expression: ");
    if (fgets(infix, sizeof(infix), stdin) != NULL) {
        infix[strcspn(infix, "\n")] = '\0';
    }

    printf("Postfix Expression:\n");
    while (infix[i] != '\0') {
        buffer = infix[i];

        if (buffer == ' ' || buffer == '\t') {
            i++; continue;
        }

        if (isdigit(buffer)) {
            while (isdigit(infix[i])) {
                printf("%c", infix[i]);
                i++;
            }
```

```c
            printf(" ");
            continue;
        } else if (buffer == '(')
            push(&s, buffer);
        else if (buffer == ')') {
            buffer = pop(&s);
            while (buffer ≠ '(' && buffer ≠ '\0') {
                printf("%c ", buffer);
                buffer = pop(&s);
            }
        } else {
            while (s.top ≠ -1 && precedence(peek(s)) ≥ precedence(buffer))
                printf("%c ", pop(&s));
            push(&s, buffer);
        }

        i++;
    }

    buffer = pop(&s);
    while (buffer ≠ '\0') {
        printf("%c", buffer);
        buffer = pop(&s);
    }

    return 0;
}
```

PROGRAM OUTPUT Execution Verification

```
Enter infix expression: (2+3)*(4-5)
Postfix Expression:
2 3 + 4 5 - *
```

PROGRAM 11 `11_postfix_evaluation.c`

Evaluation of Postfix Expression Using Stack

C SOURCE CODE 11_postfix_evaluation.c

```c
/* Implementation of Postfix Expression Evaluation */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX 50

struct Stack {
    int top;
    int items[MAX];
};

struct Stack* createStack() {
    struct Stack* stack = (struct Stack*)malloc(sizeof(struct Stack));
    stack->top = -1;
    return stack;
}

void push(struct Stack* s, int item) {
    if (s->top >= MAX - 1) {
        printf("Stack overflow\n");
        return;
    }
    s->items[++(s->top)] = item;
}

int pop(struct Stack* s) {
    if (s->top == -1) {
        return -999999;
    }
    return s->items[(s->top)--];
}

int main() {
    struct Stack* stack = createStack();
    char postfix[MAX];

    printf("Enter the postfix expression: ");
    fgets(postfix, MAX, stdin);

    // Strip newline character if present
    size_t len = strlen(postfix);
    if (len > 0 && postfix[len - 1] == '\n') {
        postfix[len - 1] = '\0';
    }

    for (int i = 0; postfix[i]; i++) {
        if (isdigit(postfix[i])) {
            push(stack, postfix[i] - '0'); // Convert char directly to int digit
        } else if (postfix[i] == ' ' || postfix[i] == '\t') {
            continue;
        } else {
            int val2 = pop(stack);
            int val1 = pop(stack);

            if (val1 == -999999 || val2 == -999999) {
                printf("Invalid postfix expression\n");
                free(stack);
                return -1;
            }

            switch (postfix[i]) {
                case '+':
```

```c
                    push(stack, val1 + val2);
                    break;
                case '-':
                    push(stack, val1 - val2);
                    break;
                case '*':
                    push(stack, val1 * val2);
                    break;
                case '/':
                    if (val2 == 0) {
                        printf("Division by zero error\n");
                        free(stack);
                        return -1;
                    }
                    push(stack, val1 / val2);
                    break;
                default:
                    printf("Invalid operator: %c\n", postfix[i]);
                    free(stack);
                    return -1;
            }
        }
    }

    printf("Result: %d\n", pop(stack));

    free(stack);
    return 0;
}

```

PROGRAM OUTPUT Execution Verification

```
Enter the postfix expression: 2 3 + 4 5 - *
Result: -5
```

PROGRAM 12 `12_single_linked_list.c`

Singly Linked List Operations

C SOURCE CODE 12_single_linked_list.c

```c
/* Implementation of Singly Linked List in C */

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *head = NULL;

// 1. Create Node / List
void create() {
    int val;
    printf("Enter data : ");
    scanf("%d", &val);

    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }
    newNode->data = val;
    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
    } else {
        struct Node *temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
    printf("Node created successfully\n");
}

// 2. Display List
void display() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    struct Node *temp = head;
    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// 3. Search Element
void search() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    int key, pos = 1, found = 0;
    printf("Enter element to search : ");
    scanf("%d", &key);

    struct Node *temp = head;
    while (temp != NULL) {
```

```c
        if (temp->data == key) {
            printf("Element %d found at position %d\n", key, pos);
            found = 1;
            break;
        }
        temp = temp->next;
        pos++;
    }
    if (!found) {
        printf("Element %d not found in the list\n", key);
    }
}

// 4. Insert at Beginning
void insertAtBeginning() {
    int val;
    printf("Enter data : ");
    scanf("%d", &val);

    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }
    newNode->data = val;
    newNode->next = head;
    head = newNode;
    printf("Node inserted successfully at beginning\n");
}

// 5. Insert at Middle (by position)
void insertAtMiddle() {
    int val, pos, i;
    printf("Enter data : ");
    scanf("%d", &val);
    printf("Enter position : ");
    scanf("%d", &pos);

    if (pos < 1) {
        printf("Invalid position!\n");
        return;
    }

    if (pos == 1) {
        struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
        if (newNode == NULL) {
            printf("Memory allocation failed!\n");
            return;
        }
        newNode->data = val;
        newNode->next = head;
        head = newNode;
        printf("Node inserted successfully at position %d\n", pos);
        return;
    }

    struct Node *temp = head;
    for (i = 1; i < pos - 1 && temp != NULL; i++) {
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Position out of range!\n");
        return;
    }

    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return;
```

```c
    }
    newNode->data = val;
    newNode->next = temp->next;
    temp->next = newNode;
    printf("Node inserted successfully at position %d\n", pos);
}

// 6. Insert at End
void insertAtEnd() {
    int val;
    printf("Enter data : ");
    scanf("%d", &val);

    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }
    newNode->data = val;
    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
    } else {
        struct Node *temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
    printf("Node inserted successfully at end\n");
}

// 7. Delete at Beginning
void deleteAtBeginning() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    struct Node *temp = head;
    head = head->next;
    printf("Deleted element: %d\n", temp->data);
    free(temp);
}

// 8. Delete at Middle (by position)
void deleteAtMiddle() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    int pos, i;
    printf("Enter position to delete : ");
    scanf("%d", &pos);

    if (pos < 1) {
        printf("Invalid position!\n");
        return;
    }

    if (pos == 1) {
        struct Node *temp = head;
        head = head->next;
        printf("Deleted element: %d\n", temp->data);
        free(temp);
        return;
    }

    struct Node *temp = head;
    for (i = 1; i < pos - 1 && temp != NULL; i++) {
```

```c
        temp = temp->next;
    }

    if (temp == NULL || temp->next == NULL) {
        printf("Position out of range!\n");
        return;
    }

    struct Node *delNode = temp->next;
    temp->next = delNode->next;
    printf("Deleted element: %d\n", delNode->data);
    free(delNode);
}

// 9. Delete at End
void deleteAtEnd() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    if (head->next == NULL) {
        printf("Deleted element: %d\n", head->data);
        free(head);
        head = NULL;
        return;
    }
    struct Node *temp = head;
    while (temp->next->next != NULL) {
        temp = temp->next;
    }
    struct Node *delNode = temp->next;
    temp->next = NULL;
    printf("Deleted element: %d\n", delNode->data);
    free(delNode);
}

int main() {
    int choice;

    do {
        printf("\n- - - Linked List Menu - - -\n");
        printf("1. Create\n");
        printf("2. Display\n");
        printf("3. Search\n");
        printf("4. Insert at Beginning\n");
        printf("5. Insert at Middle\n");
        printf("6. Insert at End\n");
        printf("7. Delete at Beginning\n");
        printf("8. Delete at Middle\n");
        printf("9. Delete at End\n");
        printf("10. Exit\n");
        printf("Enter choice : ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                create();
                break;
            case 2:
                display();
                break;
            case 3:
                search();
                break;
            case 4:
                insertAtBeginning();
                break;
            case 5:
                insertAtMiddle();
                break;
```

```
            case 6:
                insertAtEnd();
                break;
            case 7:
                deleteAtBeginning();
                break;
            case 8:
                deleteAtMiddle();
                break;
            case 9:
                deleteAtEnd();
                break;
            case 10:
                printf("Exiting ... \n");
                break;
            default:
                printf("Invalid choice! Please try again.\n");
        }
    } while (choice != 10);

    return 0;
}

```

PROGRAM OUTPUT Execution Verification

```
- - - Linked List Menu - - -
1. Create
2. Display
3. Search
4. Insert at Beginning
5. Insert at Middle
6. Insert at End
7. Delete at Beginning
8. Delete at Middle
9. Delete at End
10. Exit
Enter choice : 1
Enter data : 10
Node created successfully

- - - Linked List Menu - - -
Enter choice : 2
Linked List: 10 -> NULL

- - - Linked List Menu - - -
Enter choice : 4
Enter data : 5
Node inserted successfully at beginning

- - - Linked List Menu - - -
Enter choice : 6
Enter data : 30
Node inserted successfully at end

- - - Linked List Menu - - -
Enter choice : 5
Enter data : 20
Enter position : 3
Node inserted successfully at position 3

- - - Linked List Menu - - -
Enter choice : 2
Linked List: 5 -> 10 -> 20 -> 30 -> NULL

- - - Linked List Menu - - -
Enter choice : 3
Enter element to search : 20
Element 20 found at position 3
```

```
- - - Linked List Menu - - -
Enter choice : 8
Enter position to delete : 3
Deleted element: 20

- - - Linked List Menu - - -
Enter choice : 7
Deleted element: 5

- - - Linked List Menu - - -
Enter choice : 9
Deleted element: 30

- - - Linked List Menu - - -
Enter choice : 2
Linked List: 10 → NULL

- - - Linked List Menu - - -
Enter choice : 10
Exiting...
```

PROGRAM 13 `13_doubly_linked_list.c`

Doubly Linked List Operations

C SOURCE CODE 13_doubly_linked_list.c

```c
/* Implementation of Doubly Linked List in C */

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
};

struct Node *head = NULL;

// 1. Create Node / List
void create() {
    int val;
    printf("Enter data : ");
    scanf("%d", &val);

    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }
    newNode->data = val;
    newNode->prev = NULL;
    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
    } else {
        struct Node *temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->prev = temp;
    }
    printf("Node created successfully\n");
}

// 2. Display List
void display() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    struct Node *temp = head;
    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// 3. Search Element
void search() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    int key, pos = 1, found = 0;
    printf("Enter element to search : ");
    scanf("%d", &key);
```

```c

    struct Node *temp = head;
    while (temp ≠ NULL) {
        if (temp→data == key) {
            printf("Element %d found at position %d\n", key, pos);
            found = 1;
            break;
        }
        temp = temp→next;
        pos++;
    }
    if (!found) {
        printf("Element %d not found in the list\n", key);
    }
}

// 4. Insert at Beginning
void insertAtBeginning() {
    int val;
    printf("Enter data : ");
    scanf("%d", &val);

    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }
    newNode→data = val;
    newNode→prev = NULL;
    newNode→next = head;

    if (head ≠ NULL) {
        head→prev = newNode;
    }
    head = newNode;
    printf("Node inserted successfully at beginning\n");
}

// 5. Insert at Middle (by position)
void insertAtMiddle() {
    int val, pos, i;
    printf("Enter data : ");
    scanf("%d", &val);
    printf("Enter position : ");
    scanf("%d", &pos);

    if (pos < 1) {
        printf("Invalid position!\n");
        return;
    }

    if (pos == 1) {
        insertAtBeginning();
        return;
    }

    struct Node *temp = head;
    for (i = 1; i < pos - 1 && temp ≠ NULL; i++) {
        temp = temp→next;
    }

    if (temp == NULL) {
        printf("Position out of range!\n");
        return;
    }

    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return;
```

```
    }
    newNode->data = val;
    newNode->next = temp->next;
    newNode->prev = temp;

    if (temp->next != NULL) {
        temp->next->prev = newNode;
    }
    temp->next = newNode;

    printf("Node inserted successfully at position %d\n", pos);
}

// 6. Insert at End
void insertAtEnd() {
    int val;
    printf("Enter data : ");
    scanf("%d", &val);

    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }
    newNode->data = val;
    newNode->next = NULL;
    newNode->prev = NULL;

    if (head == NULL) {
        head = newNode;
    } else {
        struct Node *temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->prev = temp;
    }
    printf("Node inserted successfully at end\n");
}

// 7. Delete at Beginning
void deleteAtBeginning() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    struct Node *temp = head;
    head = head->next;
    if (head != NULL) {
        head->prev = NULL;
    }
    printf("Deleted element: %d\n", temp->data);
    free(temp);
}

// 8. Delete at Middle (by position)
void deleteAtMiddle() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    int pos, i;
    printf("Enter position to delete : ");
    scanf("%d", &pos);

    if (pos < 1) {
        printf("Invalid position!\n");
        return;
    }
```

```c

    if (pos == 1) {
        deleteAtBeginning();
        return;
    }

    struct Node *temp = head;
    for (i = 1; i < pos && temp ≠ NULL; i++) {
        temp = temp→next;
    }

    if (temp == NULL) {
        printf("Position out of range!\n");
        return;
    }

    if (temp→prev ≠ NULL) {
        temp→prev→next = temp→next;
    }
    if (temp→next ≠ NULL) {
        temp→next→prev = temp→prev;
    }

    printf("Deleted element: %d\n", temp→data);
    free(temp);
}

// 9. Delete at End
void deleteAtEnd() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    struct Node *temp = head;
    if (head→next == NULL) {
        printf("Deleted element: %d\n", head→data);
        free(head);
        head = NULL;
        return;
    }

    while (temp→next ≠ NULL) {
        temp = temp→next;
    }

    temp→prev→next = NULL;
    printf("Deleted element: %d\n", temp→data);
    free(temp);
}

int main() {
    int choice;

    do {
        printf("\n- - - Doubly Linked List Menu - - -\n");
        printf("1. Create\n");
        printf("2. Display\n");
        printf("3. Search\n");
        printf("4. Insert at Beginning\n");
        printf("5. Insert at Middle\n");
        printf("6. Insert at End\n");
        printf("7. Delete at Beginning\n");
        printf("8. Delete at Middle\n");
        printf("9. Delete at End\n");
        printf("10. Exit\n");
        printf("Enter choice : ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
```

```
                create();
                break;
            case 2:
                display();
                break;
            case 3:
                search();
                break;
            case 4:
                insertAtBeginning();
                break;
            case 5:
                insertAtMiddle();
                break;
            case 6:
                insertAtEnd();
                break;
            case 7:
                deleteAtBeginning();
                break;
            case 8:
                deleteAtMiddle();
                break;
            case 9:
                deleteAtEnd();
                break;
            case 10:
                printf("Exiting ... \n");
                break;
            default:
                printf("Invalid choice! Please try again.\n");
        }
    } while (choice ≠ 10);

    return 0;
}

```

PROGRAM OUTPUT Execution Verification

```
- - - Doubly Linked List Menu - - -
1. Create
2. Display
3. Search
4. Insert at Beginning
5. Insert at Middle
6. Insert at End
7. Delete at Beginning
8. Delete at Middle
9. Delete at End
10. Exit
Enter choice : 1
Enter data : 10
Node created successfully

- - - Doubly Linked List Menu - - -
Enter choice : 4
Enter data : 5
Node inserted successfully at beginning

- - - Doubly Linked List Menu - - -
Enter choice : 6
Enter data : 30
Node inserted successfully at end

- - - Doubly Linked List Menu - - -
Enter choice : 5
Enter data : 20
Enter position : 3
```

```
Node inserted successfully at position 3

- - - Doubly Linked List Menu - - -
Enter choice : 2
Linked List: 5 ⟷ 10 ⟷ 20 ⟷ 30 ⟷ NULL

- - - Doubly Linked List Menu - - -
Enter choice : 3
Enter element to search : 20
Element 20 found at position 3

- - - Doubly Linked List Menu - - -
Enter choice : 8
Enter position to delete : 3
Deleted element: 20

- - - Doubly Linked List Menu - - -
Enter choice : 7
Deleted element: 5

- - - Doubly Linked List Menu - - -
Enter choice : 9
Deleted element: 30

- - - Doubly Linked List Menu - - -
Enter choice : 2
Linked List: 10 ⟷ NULL

- - - Doubly Linked List Menu - - -
Enter choice : 10
Exiting ...
```

PROGRAM 14 `14_stack_linked_list.c`

Stack Operations Using Linked List

C SOURCE CODE 14_stack_linked_list.c

```c
/* Implementation of Stack using Linked List in C */

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *top = NULL;

void push(int val) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Stack Overflow\n");
        return;
    }
    newNode->data = val;
    newNode->next = top;
    top = newNode;
}

int pop() {
    if (top == NULL) {
        printf("Stack Underflow\n");
        return -1;
    }
    struct Node *temp = top;
    int val = temp->data;
    top = top->next;
    free(temp);
    return val;
}

void display() {
    if (top == NULL) {
        printf("Stack Underflow\n");
        return;
    }
    printf("Displaying contents:\n");
    struct Node *temp = top;
    while (temp != NULL) {
        printf("%d\n", temp->data);
        temp = temp->next;
    }
}

int main() {
    int choice, value;

    do {
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to push: ");
                scanf("%d", &value);
                push(value);
                break;
```

```c
            case 2:
                value = pop();
                if (value != -1) {
                    printf("Popped value: %d\n", value);
                }
                break;
            case 3:
                display();
                break;
            case 4:
                printf("Exiting ... \n");
                break;
            default:
                printf("Invalid choice. Please try again.\n");
        }
    } while (choice != 4);

    return 0;
}

```

PROGRAM OUTPUT Execution Verification

```
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter value to push: 10
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter value to push: 20
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Displaying contents:
20
10
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 2
Popped value: 20
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Displaying contents:
10
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 4
Exiting ...
```

PROGRAM 15 `15_queue_linked_list.c`

Queue Operations Using Linked List

C SOURCE CODE 15_queue_linked_list.c

```c
/* Implementation of Queue using Linked List in C */

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *front = NULL;
struct Node *rear = NULL;

void enqueue(int val) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Queue Overflow\n");
        return;
    }
    newNode->data = val;
    newNode->next = NULL;

    if (front == NULL) {
        front = rear = newNode;
    } else {
        rear->next = newNode;
        rear = newNode;
    }
}

int dequeue() {
    if (front == NULL) {
        printf("Queue Underflow\n");
        return -1;
    }
    struct Node *temp = front;
    int val = temp->data;
    front = front->next;

    if (front == NULL) {
        rear = NULL;
    }

    free(temp);
    return val;
}

void display() {
    if (front == NULL) {
        printf("Queue Underflow\n");
        return;
    }
    printf("Displaying Contents:\n");
    struct Node *temp = front;
    while (temp != NULL) {
        printf("%d\n", temp->data);
        temp = temp->next;
    }
}

int main() {
    int choice, value;

    do {
        printf("1. Enqueue\n");
```

```c
        printf("2. Dequeue\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to enqueue: ");
                scanf("%d", &value);
                enqueue(value);
                break;
            case 2:
                value = dequeue();
                if (value != -1) {
                    printf("Dequeued value: %d\n", value);
                }
                break;
            case 3:
                display();
                break;
            case 4:
                printf("Exiting ... \n");
                break;
            default:
                printf("Invalid choice! Please try again.\n");
        }
    } while (choice != 4);

    return 0;
}

```

PROGRAM OUTPUT Execution Verification

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 100
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 200
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Displaying Contents:
100
200
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
Dequeued value: 100
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Displaying Contents:
200
1. Enqueue
```

```
2. Dequeue
3. Display
4. Exit
Enter your choice: 4
Exiting ...
```